Software Requirements Specification (SRS)

ASTU MSJ Summer Bootcamp Management System

1. Introduction

1.1 Purpose

The Bootcamp Management System is a web-based platform for managing bootcamp activities in one centralized system. It replaces manual spreadsheets, paper attendance sheets, and scattered chat messages with a single system where admins, mentors, and students manage the entire bootcamp experience.

The system will manage:

- Students and mentors (users)
- Batches
- Attendance
- Student progress
- Assignments and submissions
- Grading and feedback
- Announcements and notifications
- Role-specific dashboards

1.2 Problem Statement

ASTU MSJ currently manages bootcamp operations manually. This creates real, recurring problems:

Problem	**Impact**
Attendance tracked on paper or spreadsheets	Time-consuming, error-prone, hard to calculate percentages
No central place for weekly progress	Mentors lose track of who is behind
Assignments submitted via chat or forms	Feedback is delayed and disorganized
Announcements sent through multiple channels	Students miss important updates
No single dashboard for students	Students cannot see their overall standing at a glance

A centralized system solves these problems by giving every role exactly the tools and visibility it needs.

1.3 Scope and Creative Freedom

This project is the official final project for the bootcamp. Student teams have three weeks to design, build, and deploy a real, usable version of this system. Sections 1 through 19 define the required foundation every team's system must satisfy these. Section 20 (Optional and Bonus Features) is intentionally open-ended: teams are encouraged to extend the system, propose their own features, and make independent design choices (data models, UI layout, extra fields, workflow details) wherever this document does not specify one explicitly.

2. Objectives

The system aims to:

- Centralize bootcamp management for admins, mentors, and students.
- Track student attendance accurately and calculate percentages automatically.
- Monitor student progress across topics/modules.
- Manage assignments, submissions, grading, and feedback.
- Centralize bootcamp announcements and notifications.
- Provide role-specific dashboards.
- Secure the system using authentication and role-based authorization.
- Demonstrate clean, professional MERN stack architecture within a three-week timeline.

3. User Roles

The system has three roles. Access is enforced through Role-Based Access Control (RBAC) on both frontend routes and backend APIs.

Role	Description	Key Responsibilities
Admin	Manages the overall bootcamp	Full control over users, mentors, students, batches, announcements; views bootcamp-wide statistics
Mentor	Manages assigned students	Attendance, progress, assignment review and grading, announcements to their batch
Student	Bootcamp participant	Views own dashboard, attendance, progress, assignments, grades, and announcements

Students can only access their own records. Mentors can only manage students assigned to them. Admins have full visibility across the system.

4. Authentication & Authorization

The system shall provide:

- Registration and Login
- Logout
- JWT-based authentication
- Password hashing with bcrypt
- Password change / reset
- Protected routes
- Role-Based Access Control (RBAC) for Admin, Mentor, and Student

- Backend authorization middleware on all protected endpoints

5. Public Landing Page

The system shall have a responsive public landing page containing:

- Hero section
- About section
- Tracks section
- Mentors section
- FAQ section
- Contact section
- Login and Registration

6. Admin Module

Admins shall be able to:

- Create, update, and delete student and mentor accounts
- Search users
- Assign roles (Student, Mentor, Admin)
- Create and manage batches
- Assign mentors to batches
- Enroll students
- Create, edit, and delete announcements
- View attendance statistics
- View assignment statistics

A full resource library and a dedicated analytics/reporting module are not required for the core system see Section 20 for these as optional extensions.

7. Mentor Module

Mentors shall be able to manage their assigned students.

7.1 Attendance

- Mark attendance
- Update attendance
- View attendance history
- View attendance percentages

7.2 Progress

- Update weekly / topic progress

- Add progress notes
- Monitor students who are falling behind

7.3 Assignments

- Create assignments
- Set deadlines
- Review submissions
- Grade assignments
- Provide feedback
- Request resubmission

8. Student Module

Students shall have a personal dashboard containing:

- Attendance percentage
- Progress summary
- Assignment status
- Average grades
- Recent announcements
- Upcoming deadlines

Students shall be able to:

- View attendance
- View progress
- Submit assignments
- View grades and feedback
- View announcements
- Manage their profile

9. Attendance Management

Attendance statuses shall include:

- Present
- Absent
- Late
- Excused

The system shall automatically calculate attendance percentage:

```
Attendance % = Present Sessions / Total Applicable Sessions x 100
```

Mentors manage attendance for their assigned students; students can only view their own attendance.

10. Progress Management

Progress shall be tracked by topic / module. Example topics:

- HTML / CSS
- JavaScript
- React
- Node.js
- Express.js
- MongoDB
- Git / GitHub

Progress statuses:

- Not Started
- In Progress
- Completed
- Needs Improvement

Mentors update progress; students view their own progress.

11. Assignment & Grading

Mentors can create assignments containing:

- Title
- Description
- Instructions
- Batch
- Deadline
- Maximum score

Students can submit:

- GitHub repository URL
- Live demo URL (optional)
- Notes

Mentors can:

- Review submissions
- Give scores
- Provide feedback
- Request resubmission

File attachments for assignments and submissions are not required for the core system, but teams may add this as a creative extension (see Section 20).

12. Announcement & Notification System

Admins and mentors can create, edit, and delete announcements. Announcements shall include:

- Title
- Content
- Target audience
- Batch
- Publish date

Students shall see relevant announcements and be notified in-app for events such as:

- New assignments
- Assignment deadlines approaching
- Grades and feedback posted
- Important announcements

A standalone notifications module is not required in-app notifications may be delivered as part of the announcement system.

13. Dashboards

13.1 Admin Dashboard

- Student count
- Mentor count
- Batch count
- Attendance statistics
- Assignment statistics
- Recent activity

13.2 Mentor Dashboard

- Assigned students
- Attendance overview
- Progress overview
- Pending assignments to grade
- At-risk students

13.3 Student Dashboard

- Attendance
- Progress
- Assignments
- Grades
- Announcements

14. Database Collections

MongoDB shall contain, at minimum, the following collections. Mongoose shall be used as the ODM.

Collection	Purpose
Users	Stores students, mentors, and admins with roles and credentials
Batches	Bootcamp batches / cohorts
Attendance	Daily attendance records per student
Progress	Weekly progress entries per student per topic
Assignments	Assignment definitions created by mentors
Submissions	Student submissions, grades, and feedback
Announcements	Announcement content, author, and timestamps

Detailed schemas are left to each team's design and are not specified here. Additional collections (for example Resources or Reports) are only needed if a team implements the optional features in Section 20.

15. API Structure

The backend shall provide REST APIs such as:

```
/api/auth
/api/users
/api/batches
/api/attendance
/api/progress
/api/assignments
/api/submissions
/api/announcements
```

All protected endpoints shall use authentication and authorization middleware.

16. Non-Functional Requirements

16.1 Secure

- JWT authentication
- bcrypt password hashing
- RBAC
- Input validation
- Protected APIs
- Secure environment variables

16.2 Responsive

The application shall work on desktop, tablet, and mobile.

16.3 Scalable

The system should support multiple batches and future bootcamp cohorts.

16.4 Maintainable

The codebase shall use modular architecture: reusable components, controllers, services, middleware, validation, and centralized error handling.

17. Recommended Architecture

```
React.js
   |
   | Axios / REST API
   v
Express.js + Node.js
   |
   | Mongoose
   v
MongoDB
```

17.1 Frontend

- React
- React Router
- Axios
- Tailwind CSS
- Protected Routes and Role Guards

17.2 Backend

- Express.js
- JWT
- bcrypt
- Mongoose
- REST API
- Middleware, Validation, and Centralized Error Handling

18. Testing

The system shall be tested for:

- Authentication and authorization
- CRUD operations
- Attendance calculations

- Assignment submission and grading
- Role restrictions
- API errors
- Responsive UI
- Security vulnerabilities

19. Deployment

```
React Frontend
     |
     v
Express.js Backend
     |
     v
MongoDB
```

Frontend deployed on Vercel; backend deployed on Render (or an equivalent platform). Environment variables shall be used for sensitive configuration such as:

- MONGO_URI
- JWT_SECRET
- PORT
- CLIENT_URL

20. Optional & Bonus Features - Creative Extensions

The features below are not required to complete the core project. They exist to give teams room to be creative once the required features in Sections 1–19 are complete and stable. Teams are also encouraged to design their own additional features beyond this list, as long as the required foundation is not compromised.

Feature	**Feature**
Resource Library (materials, links, search & filtering)	Reports & Analytics Dashboard
Leaderboard	Search & Filtering across the system
Alumni Page	PDF Reports / exports
Calendar view	Achievement Badges
Dark Mode	Student Timeline
Hall of Fame	Mentor Notes
File attachments for assignments and resources	Any other feature the team proposes

Teams should only attempt optional features after all required features are complete and stable.

21. Three-Week Development Plan

Week	Focus	Deliverables
Week 1	Foundation	Project setup, database design, authentication (register / login / JWT / RBAC), user management (Admin)
Week 2	Core Features	Attendance, Progress Tracker, Assignment Submission & Grading
Week 3	Polish & Launch	Announcements, Student Dashboard, Admin Dashboard, Mentor Dashboard, Landing Page, Testing, Deployment

Teams should aim to have a working, deployed MVP by the end of Week 2, leaving Week 3 for dashboards, polish, bug fixes, and deployment.

22. Acceptance Criteria

The system is considered complete when:

- All three roles can securely log in.
- Role-based permissions work correctly.
- Admins can manage students, mentors, and batches.
- Mentors can manage attendance and progress for their assigned students.
- Students can view their academic information.
- Students can submit assignments.
- Mentors can grade submissions and provide feedback.
- Announcements can be published and are visible to the correct audience.
- Dashboards display accurate information for each role.
- Unauthorized users cannot access restricted data.
- The system works responsively on desktop and mobile.
- The application is successfully deployed.

23. Final Technology Stack

Layer	Technology
Frontend	React.js
Routing	React Router
Styling	Tailwind CSS
API Communication	Axios

Layer	Technology
Backend	Node.js + Express.js
Database	MongoDB
ODM	Mongoose
Authentication	JWT
Password Security	bcrypt
Authorization	RBAC
Architecture	REST API
Deployment	Vercel (frontend) / Render (backend), or equivalent

---

This document is the official reference for all teams building the ASTU MSJ Summer Bootcamp Management System. Build with clarity, focus, and pride this system may serve future bootcamp cohorts. Remember This specification by no means a complete document feel free to to add your own feature and creativity to it